



Universidade Estácio de Sá (UNESA)

Campus Estácio - Castelo - Belo Horizonte - MG

Desenvolvimento Full Stack

RELATÓRIO DO 1º PROCEDIMENTO DA MISSÃO PRÁTICA

Nível 2: Vamos Manter as Informações? (RPG0015) - Mundo 3

Turma: 9002 - Semestre: 2025.1

Aluno: Ivan de Ávila Carvalho Fleury Mortimer

1º Procedimento | Criando o Banco de Dados

1. Objetivo da Prática

Este relatório tem como objetivo apresentar a modelagem e implementação de um banco de dados relacional para controle de operações de compra e venda de produtos, conforme o roteiro da prática proposto na disciplina. As estruturas do banco foram criadas utilizando a linguagem Transact-SQL no SQL Server, com base em um modelo lógico de dados elaborado com o apoio de uma ferramenta de modelagem (DbSchema).

A razão de eu ter utilizado o aplicativo DbSchema ao invés do aplicativo DBDesigner Fork para a geração dos modelos lógico e físico do banco de dados relacionais (que me auxiliariam na geração do código T-SQL que seria rodado no SQL Server), é que meu computador é um MacBook Air do ano 2018 (com processador Intel), que opera com o sistema operacional macOS Sonoma, e não existe versão do aplicativo DBDesigner Fork que rode em um sistema macOS. Minha única alternativa, se eu realmente quisesse utilizar o DBDesigner Fork em meu computador Mac, seria instalar uma máquina virtual (como, por exemplo, o VirtualBox da Oracle), instalar um sistema operacional Windows dentro dessa máquina virtual e rodar o aplicativo DBDesigner Fork dentro desse sistema operacional "embutido" na minha máquina. Porém dado as baixas capacidades de processamento, de memória (RAM) armazenagem, do meu computador, essa opção se tornou inviável. (Estou trabalhando com um computador MacBook Air fabricado em 2018, com processador Intel Core i5 Dual-Core com clock máximo de 1,6 GHz, com 8 GB de memória RAM (2133 MHz LPDDR3), e 121,02 GB de armazenamento em SSD, rodando o sistema operacional macOS Sonoma, versão 14.7.5). Por isso, peço desculpas por esse desvio das instruções exatas para a execução do trabalho. Foi um desvio necessário perante os recursos que eu tinha a minha disposição. Pela mesma razão, tive que utilizar o aplicativo Azure Data Studio (ADS) para gerenciar o banco de dados relacional SQL Server, ao invés do aplicativo SQL Server Management Studio (SSMS) que foi sugerido nas instruções da Missão Prática. Apesar de o aplicativo SQL Server não possuir versão nativa para o sistema operacional macOS, como se trata de um aplicativo leve, sem interface gráfica, consegui rodá-lo no macOS dentro de um container criado pelo aplicativo Docker.

2. Script T-SQL de Criação do Banco de Dados

A seguir, é apresentado o script T-SQL utilizado para criação do banco de dados relacional Loja_DB e suas tabelas associadas. Esse script foi gerado com a ajuda do aplicativo DbSchema (onde foram gerados o modelos lógico relacional e o modelo físico baseado no banco de dados relacional SQL Server, baseado no modelo lógico relacional anterior), e depois foi modificado e ajustado, conforme as especificações da atividade prática, no aplicativo Azure Data Studio conectado a um banco de dados SQL Server. Arquivo do script: *SQL_Script_CREATE_DATABASE_Loja_DB_and_SCHEMA_dbo_with_tables_and_triggers_01.sql*

```

-- 1. Criação do banco de dados se não existir
IF DB_ID('Loja_DB') IS NULL EXECUTE('CREATE DATABASE [Loja_DB];');
GO

-- 2. Troca para o banco Loja_DB
USE [Loja_DB];
GO

-- 3. Criação do esquema dbo se não existir
IF SCHEMA_ID('dbo') IS NULL EXECUTE('CREATE SCHEMA [dbo];');
GO

-- 4. Criação do LOGIN e USER para 'loja'
IF NOT EXISTS (SELECT * FROM sys.sql_logins WHERE name = 'loja')
BEGIN
    CREATE LOGIN loja WITH PASSWORD = 'loja', CHECK_POLICY = OFF;
END;
GO

USE [Loja_DB];
GO

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name =
'loja')
BEGIN
    CREATE USER loja FOR LOGIN loja;
    EXEC sp_addrolemember 'db_owner', 'loja';
END;
GO

-- 5. Criação da SEQUENCE para ID de pessoa
IF OBJECT_ID('dbo.Seq_idPessoa', 'SO') IS NULL
BEGIN
    CREATE SEQUENCE dbo.Seq_idPessoa
        START WITH 1
        INCREMENT BY 1;
END;
GO

-- 6. Tabela Pessoa
CREATE TABLE dbo.Pessoa (
    idPessoa INT NOT NULL PRIMARY KEY DEFAULT NEXT VALUE FOR
dbo.Seq_idPessoa,
    nome VARCHAR(255) NOT NULL,
    cep CHAR(8) NOT NULL,
    logradouro VARCHAR(100) NOT NULL,
    numero VARCHAR(20) NOT NULL,
    complemento VARCHAR(50) NULL,
    bairro VARCHAR(50) NULL,
    cidade VARCHAR(50) NOT NULL,
    estado CHAR(2) NOT NULL,
    telefone CHAR(11) NOT NULL,
    email VARCHAR(255) NOT NULL,
    idUsuario INT NOT NULL
);
GO

```

```

-- 7. Tabelas Pessoa_Fisica e Pessoa_Juridica (herança 1:1)
CREATE TABLE dbo.Pessoa_Fisica (
    idPessoaFisica INT NOT NULL PRIMARY KEY,
    cpf CHAR(11) NOT NULL UNIQUE
);
GO

CREATE TABLE dbo.Pessoa_Juridica (
    idPessoaJuridica INT NOT NULL PRIMARY KEY,
    cnpj CHAR(14) NOT NULL UNIQUE,
    idPessoa INT NULL
);
GO

-- 8. Tabela Usuario
CREATE TABLE dbo.Usuario (
    idUsuario INT IDENTITY(1,1) PRIMARY KEY,
    loginUsuario VARCHAR(255) NOT NULL UNIQUE,
    senhaUsuario VARCHAR(255) NOT NULL
);
GO

-- 9. Tabela Produto
CREATE TABLE dbo.Produto (
    idProduto INT IDENTITY(1,1) PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    quantidade INT NOT NULL,
    precoVenda NUMERIC(18,2) NOT NULL
);
GO

-- 10. Tabela Movimento_de_Compra
CREATE TABLE dbo.Movimento_de_Compra (
    idMovimentoDeCompra INT IDENTITY(1,1) PRIMARY KEY,
    idUsuario INT NOT NULL,
    idPessoaJuridica INT NOT NULL,
    idProduto INT NOT NULL,
    quantidadeDeProdutos INT NOT NULL,
    precoUnitario NUMERIC(18,2) NOT NULL
);
GO

-- 11. Tabela Movimento_de_Venda
CREATE TABLE dbo.Movimento_de_Venda (
    idMovimentoDeVenda INT IDENTITY(1,1) PRIMARY KEY,
    idUsuario INT NOT NULL,
    idPessoaFisica INT NOT NULL,
    idProduto INT NOT NULL,
    quantidadeDeProdutos INT NOT NULL,
    precoVenda NUMERIC(18,2) NOT NULL
);
GO

-- 12. Foreign Keys
ALTER TABLE dbo.Pessoa_Fisica
    ADD CONSTRAINT fk_Pessoa_Fisica_Pessoa FOREIGN KEY
(idPessoaFisica) REFERENCES dbo.Pessoa(idPessoa);
GO

```

```

-- 12. Foreign Keys
ALTER TABLE dbo.Pessoa_Fisica
    ADD CONSTRAINT fk_Pessoa_Fisica_Pessoa FOREIGN KEY
(idPessoaFisica) REFERENCES dbo.Pessoa(idPessoa);
GO

ALTER TABLE dbo.Pessoa_Juridica
    ADD CONSTRAINT fk_Pessoa_Juridica_Pessoa FOREIGN KEY
(idPessoaJuridica) REFERENCES dbo.Pessoa(idPessoa);
GO

ALTER TABLE dbo.Movimento_de_Compra
    ADD CONSTRAINT fk_Compra_Usuario FOREIGN KEY (idUsuario)
REFERENCES dbo.Usuario(idUsuario),
    CONSTRAINT fk_Compra_Pessoa_Juridica FOREIGN KEY
(idPessoaJuridica) REFERENCES
dbo.Pessoa_Juridica(idPessoaJuridica),
    CONSTRAINT fk_Compra_Produto FOREIGN KEY
(idProduto) REFERENCES dbo.Produto(idProduto);
GO

ALTER TABLE dbo.Movimento_de_Venda
    ADD CONSTRAINT fk_Venda_Usuario FOREIGN KEY (idUsuario)
REFERENCES dbo.Usuario(idUsuario),
    CONSTRAINT fk_Venda_Pessoa_Fisica FOREIGN KEY
(idPessoaFisica) REFERENCES dbo.Pessoa_Fisica(idPessoaFisica),
    CONSTRAINT fk_Venda_Produto FOREIGN KEY (idProduto)
REFERENCES dbo.Produto(idProduto);
GO

-- 13. Trigger para sincronizar precoVenda após INSERT
IF OBJECT_ID('dbo.trg_SetPrecoVenda', 'TR') IS NOT NULL DROP
TRIGGER dbo.trg_SetPrecoVenda;
GO
CREATE TRIGGER dbo.trg_SetPrecoVenda
ON dbo.Movimento_de_Venda
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE mv
    SET mv.precoVenda = p.precoVenda
    FROM dbo.Movimento_de_Venda mv
    JOIN inserted i ON mv.idMovimentoDeVenda = i.idMovimentoDeVenda
    JOIN dbo.Produto p ON p.idProduto = i.idProduto;
END;
GO

-- 14. Trigger para validação de precoVenda (não pode divergir do
Produto)
IF OBJECT_ID('dbo.trg_ValidarPrecoVenda', 'TR') IS NOT NULL DROP
TRIGGER dbo.trg_ValidarPrecoVenda;
GO
CREATE TRIGGER dbo.trg_ValidarPrecoVenda
ON dbo.Movimento_de_Venda
INSTEAD OF INSERT, UPDATE
AS

```

```

-- 12. Foreign Keys
ALTER TABLE dbo.Pessoa_Fisica
    ADD CONSTRAINT fk_Pessoa_Fisica_Pessoa FOREIGN KEY
(idPessoaFisica) REFERENCES dbo.Pessoa(idPessoa);
GO

ALTER TABLE dbo.Pessoa_Juridica
    ADD CONSTRAINT fk_Pessoa_Juridica_Pessoa FOREIGN KEY
(idPessoaJuridica) REFERENCES dbo.Pessoa(idPessoa);
GO

ALTER TABLE dbo.Movimento_de_Compra
    ADD CONSTRAINT fk_Compra_Usuario FOREIGN KEY (idUsuario)
REFERENCES dbo.Usuario(idUsuario),
    CONSTRAINT fk_Compra_Pessoa_Juridica FOREIGN KEY
(idPessoaJuridica) REFERENCES
dbo.Pessoa_Juridica(idPessoaJuridica),
    CONSTRAINT fk_Compra_Produto FOREIGN KEY
(idProduto) REFERENCES dbo.Produto(idProduto);
GO

ALTER TABLE dbo.Movimento_de_Venda
    ADD CONSTRAINT fk_Venda_Usuario FOREIGN KEY (idUsuario)
REFERENCES dbo.Usuario(idUsuario),
    CONSTRAINT fk_Venda_Pessoa_Fisica FOREIGN KEY
(idPessoaFisica) REFERENCES dbo.Pessoa_Fisica(idPessoaFisica),
    CONSTRAINT fk_Venda_Produto FOREIGN KEY (idProduto)
REFERENCES dbo.Produto(idProduto);
GO

-- 13. Trigger para sincronizar precoVenda após INSERT
IF OBJECT_ID('dbo.trg_SetPrecoVenda', 'TR') IS NOT NULL DROP
TRIGGER dbo.trg_SetPrecoVenda;
GO
CREATE TRIGGER dbo.trg_SetPrecoVenda
ON dbo.Movimento_de_Venda
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE mv
    SET mv.precoVenda = p.precoVenda
    FROM dbo.Movimento_de_Venda mv
    JOIN inserted i ON mv.idMovimentoDeVenda = i.idMovimentoDeVenda
    JOIN dbo.Produto p ON p.idProduto = i.idProduto;
END;
GO

```

```

-- 14. Trigger para validação de precoVenda (não pode divergir do
Produto)
IF OBJECT_ID('dbo.trg_ValidarPrecoVenda', 'TR') IS NOT NULL DROP
TRIGGER dbo.trg_ValidarPrecoVenda;
GO
CREATE TRIGGER dbo.trg_ValidarPrecoVenda
ON dbo.Movimento_de_Venda
INSTEAD OF INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN dbo.Produto p ON i.idProduto = p.idProduto
        WHERE i.precoVenda <> p.precoVenda
    )
    BEGIN
        RAISERROR('O precoVenda informado não corresponde ao
precoVenda do Produto.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END;

    IF EXISTS (SELECT * FROM inserted)
    BEGIN
        MERGE dbo.Movimento_de_Venda AS target
        USING inserted AS source
        ON target.idMovimentoDeVenda = source.idMovimentoDeVenda
        WHEN MATCHED THEN
            UPDATE SET
                idUsuario = source.idUsuario,
                idPessoaFisica = source.idPessoaFisica,
                idProduto = source.idProduto,
                quantidadeDeProdutos = source.quantidadeDeProdutos,
                precoVenda = source.precoVenda
        WHEN NOT MATCHED THEN
            INSERT (idUsuario, idPessoaFisica, idProduto,
quantidadeDeProdutos, precoVenda)
            VALUES (source.idUsuario, source.idPessoaFisica,
source.idProduto, source.quantidadeDeProdutos, source.precoVenda);
    END
END;
GO

```

3. Os Resultados da Execução dos Códigos

A execução do script T-SQL resultou na criação do banco de dados relacional 'Loja_DB', no esquema 'dbo', contendo as seguintes tabelas, restrições e objetos complementares, conforme o modelo lógico definido:

Tabela: Pessoa

- Descrição: Tabela base para armazenar os dados de identificação, localização e contato de qualquer pessoa (física ou jurídica).
- Chave Primária: idPessoa (gerada via SEQUENCE: NEXT VALUE FOR dbo.Seq_idPessoa).
- Campos principais: nome, endereço, cidade, estado, telefone, email, idUsuario.

Tabela: Pessoa_Fisica

- Descrição: Representa uma especialização de Pessoa para pessoas físicas.
- Chave Primária: idPessoaFisica.
- Restrições: cpf com restrição UNIQUE.
- Chave Estrangeira: fk_Pessoa_Fisica_Pessoa → Pessoa(idPessoa).

Tabela: Pessoa_Juridica

- Descrição: Representa uma especialização de Pessoa para pessoas jurídicas.
- Chave Primária: idPessoaJuridica.
- Restrições: cnpj com restrição UNIQUE.
- Chave Estrangeira: fk_Pessoa_Juridica_Pessoa → Pessoa(idPessoa).

Tabela: Usuario

- Descrição: Contém os dados de login dos operadores do sistema.
- Chave Primária: idUsuario (IDENTITY).
- Restrições: loginUsuario com restrição UNIQUE.

Tabela: Produto

- Descrição: Armazena os produtos disponíveis para compra e venda.
- Chave Primária: idProduto (IDENTITY).
- Campos: nome, quantidade, precoVenda.

Tabela: Movimento_de_Compra

- Descrição: Registra operações de compra feitas por operadores de uma pessoa jurídica.
- Chave Primária: idMovimentoDeCompra (IDENTITY).
- Chaves Estrangeiras:
 - fk_Compra_Usuario → Usuario(idUsuario)
 - fk_Compra_Pessoa_Juridica → Pessoa_Juridica(idPessoaJuridica)
 - fk_Compra_Produto → Produto(idProduto)

Tabela: Movimento_de_Venda

- Descrição: Registra operações de venda feitas por operadores a uma pessoa física.
- Chave Primária: idMovimentoDeVenda (IDENTITY).
- Chaves Estrangeiras:
 - fk_Venda_Usuario → Usuario(idUsuario)
 - fk_Venda_Pessoa_Fisica → Pessoa_Fisica(idPessoaFisica)
 - fk_Venda_Produto → Produto(idProduto)

Objeto: SEQUENCE dbo.Seq_idPessoa

- Finalidade: Geração automática de IDs únicos para a tabela Pessoa.
- Incremento automático de 1 em 1.

Trigger: trg_SetPrecoVenda

- Tipo: AFTER INSERT
- Finalidade: Atualiza automaticamente o campo precoVenda na tabela Movimento_de_Venda com o valor atual do Produto correspondente.

Trigger: trg_ValidarPrecoVenda

- Tipo: INSTEAD OF INSERT, UPDATE
- Finalidade: Impede operações em Movimento_de_Venda caso o precoVenda informado não corresponda ao precoVenda do Produto.

- Comportamento: Exibe erro (RAISERROR) e executa ROLLBACK da transação em caso de inconsistência.

4. Análise e Conclusão

a) Como são implementadas as diferentes cardinalidades, basicamente 1x1, 1xN ou NxN, em um banco de dados relacional?

→ As cardinalidades são implementadas por meio de relacionamentos com chaves estrangeiras. No caso de 1x1, uma das tabelas referencia a outra com uma chave única. No caso de 1xN, a tabela 'N' possui uma chave estrangeira referenciando a tabela '1'. Em relacionamentos NxN, geralmente é criada uma tabela intermediária com duas chaves estrangeiras para associar os registros entre as duas tabelas principais.

b) Que tipo de relacionamento deve ser utilizado para representar o uso de herança em bancos de dados relacionais?

→ O uso de herança é representado normalmente através de relacionamentos 1x1 entre uma tabela base (por exemplo, Pessoa) e suas especializações (Pessoa_Fisica e Pessoa_Juridica). Cada especialização contém a chave primária da base como chave estrangeira e primária simultaneamente.

c) Como o SQL Server Management Studio permite a melhoria da produtividade nas tarefas relacionadas ao gerenciamento do banco de dados?

→ O SQL Server Management Studio (SSMS) oferece uma interface gráfica amigável que facilita a criação de objetos de banco de dados, execução de scripts SQL, visualização de dados e gerenciamento de permissões, promovendo agilidade e organização no desenvolvimento e administração de bancos de dados. Como alternativa multiplataforma, o Azure Data Studio também foi utilizado com sucesso no macOS.

5. Repositório de Código

O projeto foi versionado e está disponível no GitHub no seguinte link:

https://github.com/ivanmortimer/Missao_Pratica_N2_M3_Vamos_Manter_as_Informacoes